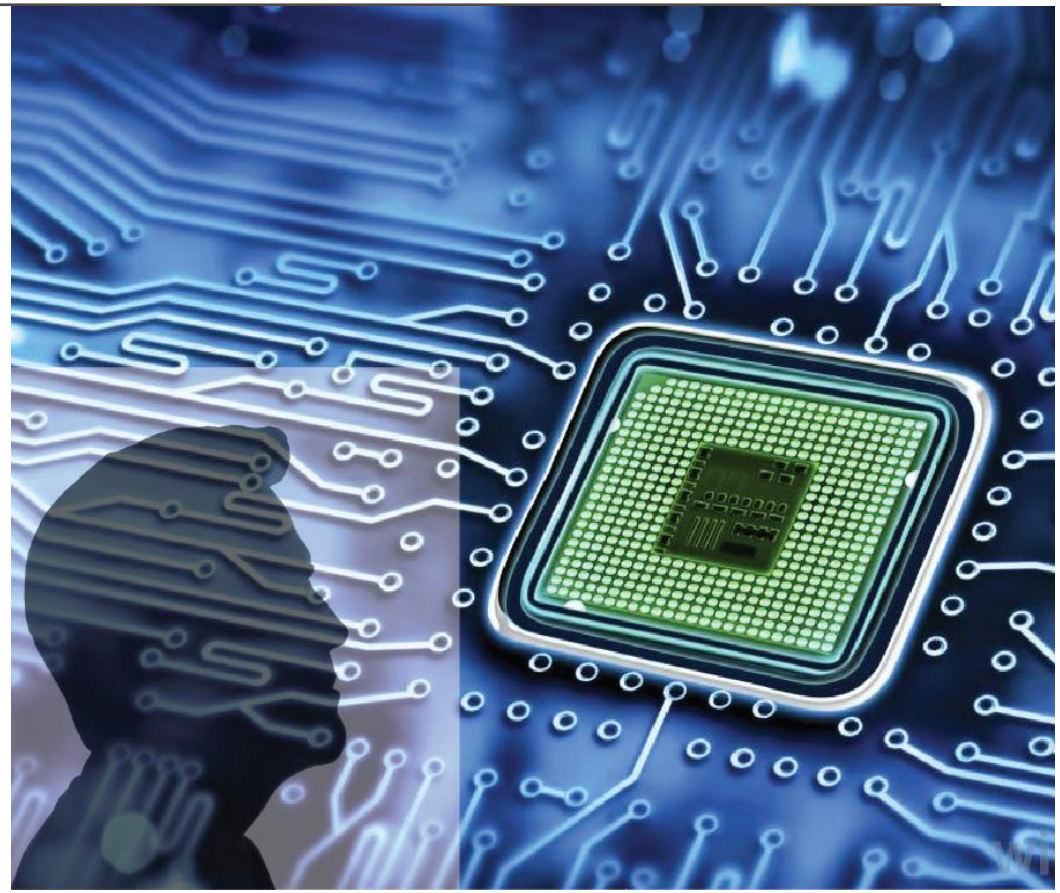
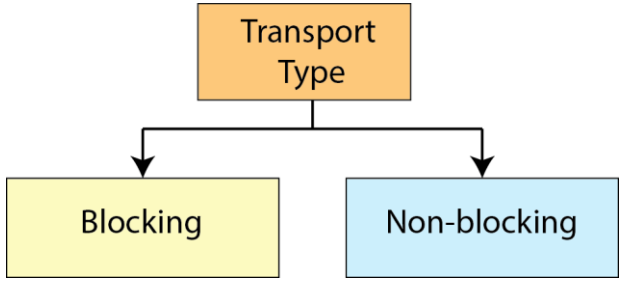
ADVANCED DIGITAL DESIGN AND VERIFICATION

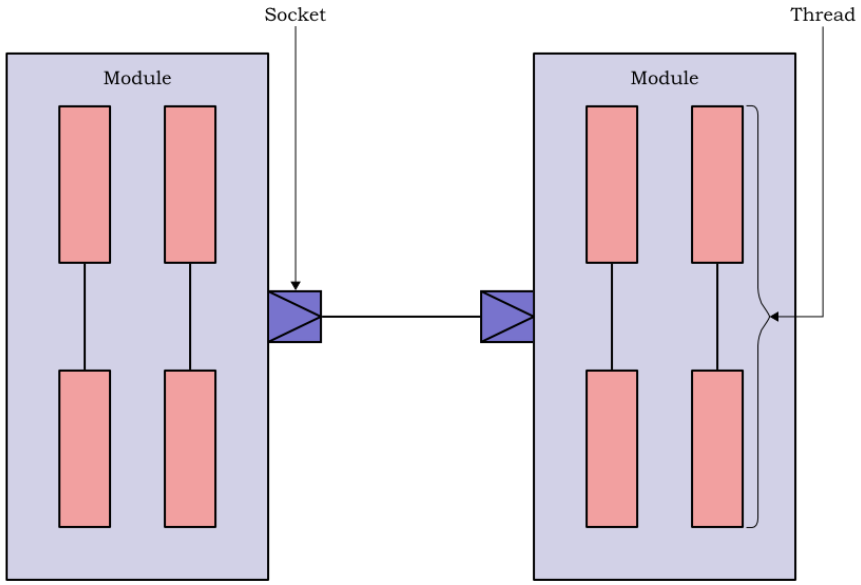
Week-4 Lecture-1
TLM



Sneh Saurabh
2nd September, 2025

TLM: Types of Transport Function





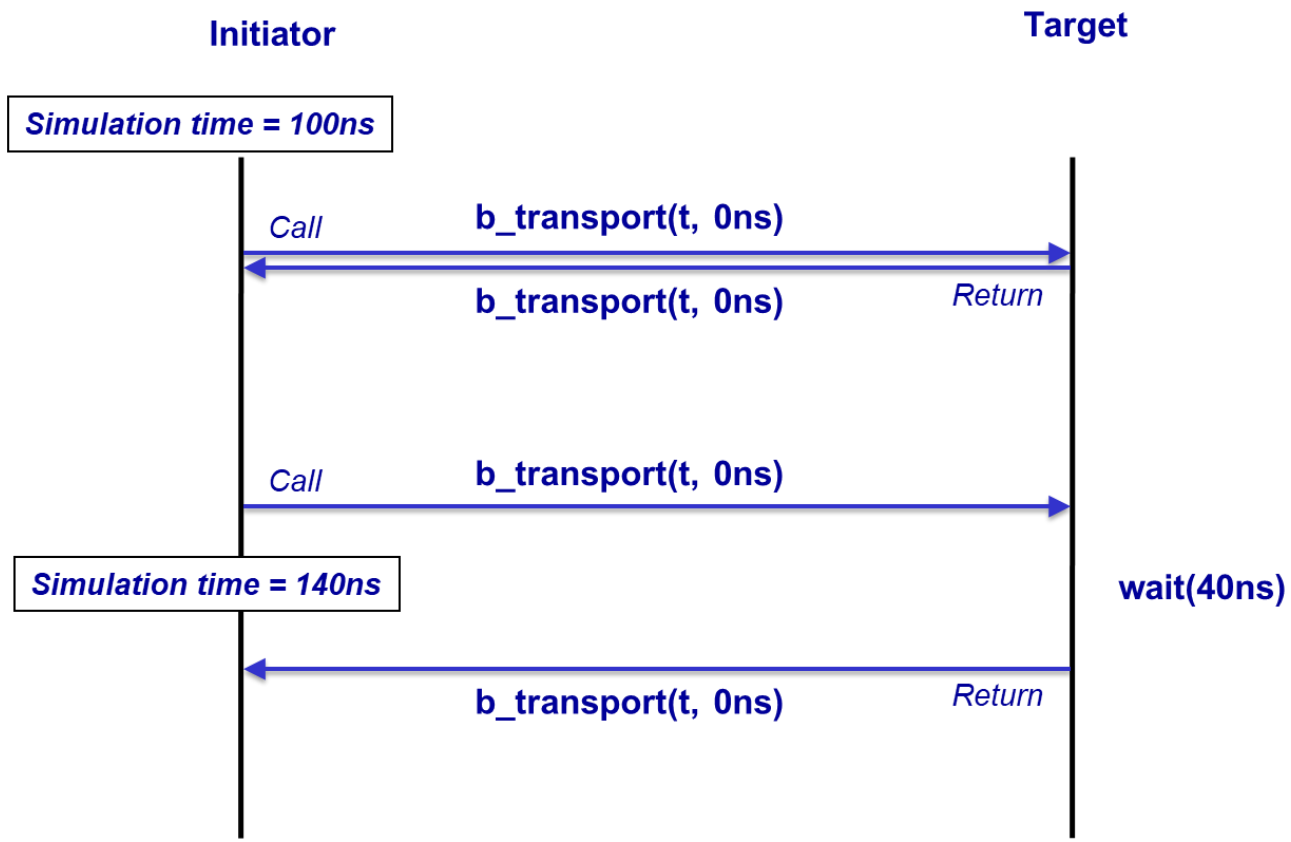
Blocking

- Initiator thread starts transaction, it is received by the target thread
- Target thread processes the request and then returns the result.
- Until the transaction has been processed by requestor and released the initiator thread is **blocked**
- Typically, **Loosely-timed** models are implemented with a blocking interface

Non-blocking:

- Target thread immediately returns before processing the request
- Typically, **Approximately-timed** models are implemented with a non-blocking interface

TLM: Blocking Transport



Mechanism:

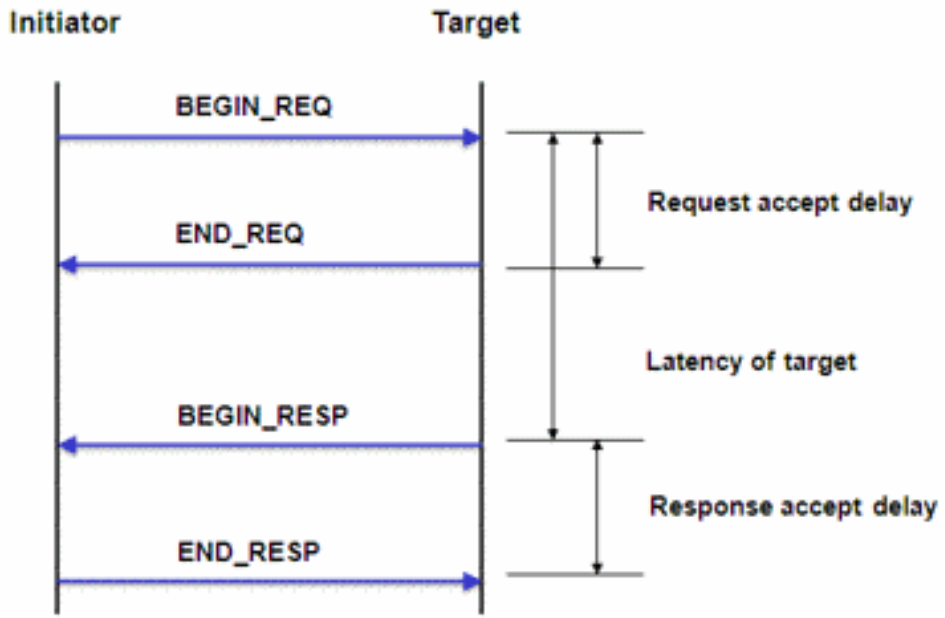
- May return immediately
- May yield control to the scheduler and only return to the initiator at a later point in simulation time
- While initiator thread may be blocked, another thread in the initiator may be permitted to call (as per protocol)

TLM: Non-Blocking Transport

Mechanism:

- Transaction to execute in multiple phases
- Transaction is passed back-and-forth between initiator and target

- Protocol can add more phases
- Caller should yield control to the scheduler and expect to receive a call on the opposite path
- Processes are regularly yielding control to the scheduler in order to allow simulation time to advance
  - Approximately-timed coding style is expected to simulate a lot more slowly than the Loosely-timed coding style



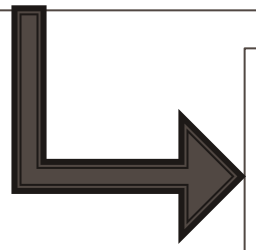
TLM: Convenience Sockets

- TLM 2.0: requires the user to derive their own classes from sockets, so that those sockets can then implement the interfaces.
- Modules then instantiate these derived sockets and use the bind function to connect them to sockets on other modules.
- Need to define new derived classes for sockets is an unnecessary layer of complexity for simple modeling.

Convenience Sockets:

- For such uses, the TLM 2.0 standard defines a number of convenience sockets which can be instantiated directly by modules, and which specify their interface functions as callbacks.

Transaction-level Model



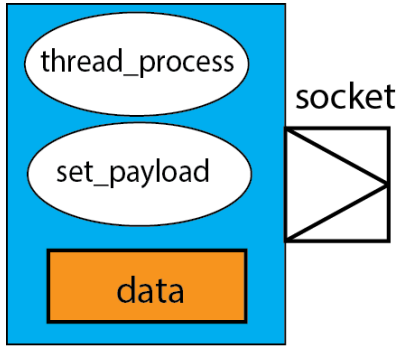
Examples

TLM Example: Setup

```
// Needed for the simple_target_socket
#define SC_INCLUDE_DYNAMIC_PROCESSES

#include "systemc"
#include "tlm.h"
#include "tlm_utils/simple_initiator_socket.h"
#include "tlm_utils/simple_target_socket.h"

using namespace sc_core;
using namespace sc_dt;
using namespace std;
```

- Necessary to define the macro SC_INCLUDE_DYNAMIC_PROCESSES (because socket spawns dynamic processes)
- To use SystemC TLM: need to include “tlm.h”
- Need to include tlm_utils: simple sockets are declared here
- Namespaces

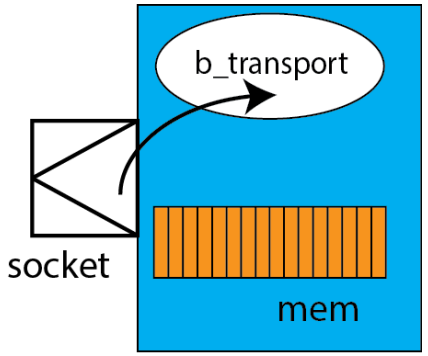
TLM Example: Initiator



- Simple socket declared: typename is the class of Initiator
  - Other default arguments: width and transaction type (tlm_generic_payload)
- Simple sockets are derived from tlm_initiator_socket and tlm_target_socket

```
// Initiator module generating generic payload transactions
SC_MODULE(Initiator)
{
  // TLM-2 socket, defaults to 32-bits wide, base protocol
  tlm_utils::simple_initiator_socket<Initiator> socket;

  SC_CTOR(Initiator)
  : socket("socket")  // Construct and name socket
  {
    SC_THREAD(thread_process);
  }
  void thread_process();
  void set_payload(tlm::tlm_generic_payload* trans,
    tlm::tlm_command cmd, int addr);

  // Internal data buffer used by initiator with generic payload
  int data;
};
```

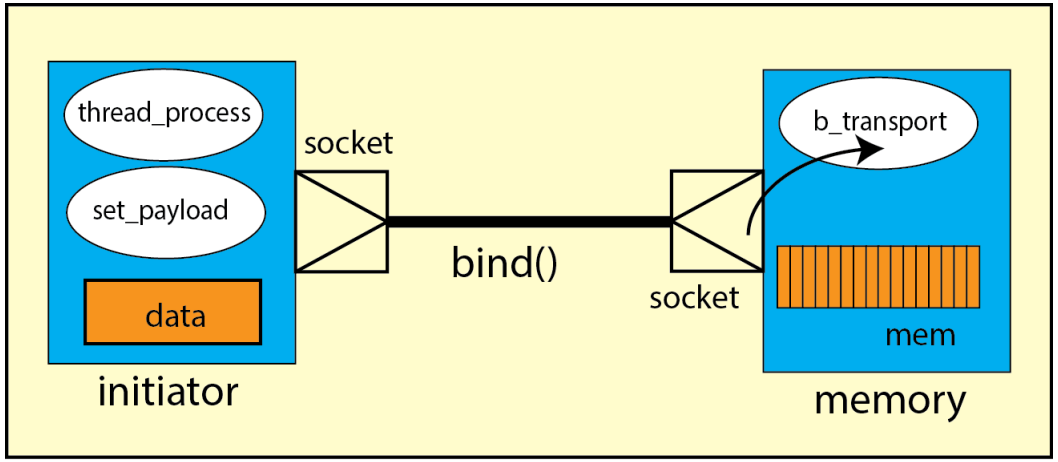
TLM Example: Target



- Initiator to communicate with the target using blocking transport
- Target implements b_transport
- Callback registered with the socket
- mem is a data member
- mem initialized to random values

```
SC_MODULE(Memory)
{
  // TLM-2 socket, defaults to 32-bits wide, base protocol
  tlm_utils::simple_target_socket<Memory> socket;

  enum { SIZE = 256 };

  SC_CTOR(Memory)
  : socket("socket")
  {
    // Register callback for incoming b_transport interface method call
    socket.register_b_transport(this, &Memory::b_transport);

    // Initialize memory with random data
    for (int i = 0; i < SIZE; i++)
      mem[i] = 0xAA000000 | (rand() % 256);
  }

  // TLM-2 blocking transport method
  virtual void b_transport( tlm::tlm_generic_payload& trans, sc_time& delay);

  int mem[SIZE];
};
```

TLM Example: Top



Top

```
SC_MODULE(Top)
{
  Initiator *initiator;
  Memory    *memory;

  SC_CTOR(Top)
  {
    // Instantiate components
    initiator = new Initiator("initiator");
    memory    = new Memory   ("memory");

    // One initiator is bound directly to
    // one target with no intervening bus
    // Bind initiator socket to target socket
    initiator->socket.bind( memory->socket );
  }
};
```

- Instantiates one initiator and one memory
- Binds the initiator socket on the initiator to the target socket on the target memory.
- The sockets encapsulate ports and exports for both directions of communication.
- One initiator socket is always bound to one target socket.

TLM Example: Payload (1)

```
void Initiator::set_payload(tlm::tlm_generic_payload* trans,
    tlm::tlm_command cmd, int addr)
{
        // Initialize 8 out of the 10 attributes,
        // byte_enable_length and extensions being unused
        trans->set_command( cmd );
        trans->set_address( addr );
        trans->set_data_ptr( reinterpret_cast<unsigned char*>(&data) )
        trans->set_data_length( 4 );
        // = data_length to indicate no streaming
        trans->set_streaming_width( 4 );
        trans->set_byte_enable_ptr( 0 ); // 0 indicates unused
                                         // Mandatory initial value
        trans->set_dmi_allowed( false );
        trans->set_response_status( tlm::TLM_INCOMPLETE_RESPONSE );


}
```

tlm_generic_payload

- Has fields of command, address, and data
- Why address for buses?
  - ➢ Allowed memory-mapped busses modelling

Memory-mapped bus

- A memory-mapped bus uses an address to locate the memories and IO/peripherals attached to a bus in a uniform manner
- Can be used as a general-purpose transaction
  - ➢ When we are not concerned with the exact details of any particular bus protocol
- Can be used as the basis for modeling a wide range of specific protocols
- Offers **inter-operability**

TLM Example: Payload (2)

```
void Initiator::set_payload(tlm::tlm_generic_payload* trans,
    tlm::tlm_command cmd, int addr)
{
        // Initialize 8 out of the 10 attributes,
        // byte_enable_length and extensions being unused
        trans->set_command( cmd );
        trans->set_address( addr );
        trans->set_data_ptr( reinterpret_cast<unsigned char*>(&data) )
        trans->set_data_length( 4 );
        // = data_length to indicate no streaming
        trans->set_streaming_width( 4 );
        trans->set_byte_enable_ptr( 0 ); // 0 indicates unused
                                         // Mandatory initial value
        trans->set_dmi_allowed( false );
        trans->set_response_status( tlm::TLM_INCOMPLETE_RESPONSE );


}
```

Data

- Data pointer attribute points to a data buffer within the initiator: owned by the initiator
- Data length attribute gives the length of the data array in bytes (4 bytes in this case)

Byte Enable

- Mask the input data so that only specific bytes of data are written (unused in this case)

DMI Allowed

- Set by the target to indicate that it supports DMI

Response: set by the target

Streaming

- Streaming width attribute specifies the width of a streaming burst where the address repeats itself.
- For a non-streaming transaction, the streaming width should equal the data length, as is the case here.

TLM Example: Initiator Thread

```
void Initiator::thread_process()
{
    // TLM-2 generic payload transaction, reused across calls to b_transport
    tlm::tlm_generic_payload* trans = new tlm::tlm_generic_payload;
    sc_time delay = sc_time(10, SC_NS);

    // Generate a random sequence of reads and writes
    for (int i = 32; i < 96; i += 4)
    {

      tlm::tlm_command cmd = static_cast<tlm::tlm_command>(rand() % 2);
      if (cmd == tlm::TLM_WRITE_COMMAND) data = 0xFF000000 | i;

      set_payload(trans, cmd, i);

      // Blocking transport call
      socket->b_transport( *trans, delay );

      // Initiator obliged to check response status and delay
      if ( trans->is_response_error() )
        SC_REPORT_ERROR("TLM-2", "Response error from b_transport");

      cout << "trans = { " << (cmd ? 'W' : 'R') << ", " << hex << i
           << " } , data = " << hex << data
           << " at time " << sc_time_stamp()
           << " delay = " << delay << endl;
      // Realize the delay annotated onto the transport call
      wait(delay);
    }
}
```

- Generate a stream of generic payload transactions
  - ➢ Each with a delay of 10 ns
- Sends it to the socket
  - ➢ Also carries delay: to be added to the current simulation time to determine the time at which the transaction is to be processed
- Receives response
- Shows the value of data (new value after read, if “R”)

TLM Example: Target Response

```
 // TLM-2 blocking transport method
void Memory::b_transport( tlm::tlm_generic_payload& trans, sc_time& delay )
{
    tlm::tlm_command cmd = trans.get_command();
    sc_dt::uint64    adr = trans.get_address() / 4;
    unsigned char*   ptr = trans.get_data_ptr();
    unsigned int     len = trans.get_data_length();
    unsigned char*   byt = trans.get_byte_enable_ptr();
    unsigned int     wid = trans.get_streaming_width();

    // Check address range and check for unsupported features,
    if (adr >= sc_dt::uint64(SIZE) || byt != 0 || len > 4 || wid < len)
      SC_REPORT_ERROR("TLM-2",
        "Target does not support given generic payload transaction");

    // Implement read and write commands
    if ( cmd == tlm::TLM_READ_COMMAND )
      memcpy(ptr, &mem[adr], len);
    else if ( cmd == tlm::TLM_WRITE_COMMAND )
      memcpy(&mem[adr], ptr, len);

    // Set response status to indicate successful completion
    trans.set_response_status( tlm::TLM_OK_RESPONSE );
  }
```

- Extracts attributes from the payload
- Check compatibility of the target with the initiator's request
- For write command: the data will be copied from the data array (of size len) to the target
- For read command: data copied from the target to the data array (of size len)
- Endianness assumed to be of the host machine for both initiator and target (so memcpy can be used)
- Respond: TLM_OK_RESPONSE
- Ignores delay and does not wait

TLM Example: Result

- In this case, blocking transport method is only modeling the functionality of the target (no delay)
- Delay annotation only on the side of initiator

Why such an approach?

- Focus is to compute the functionality of the initiator and target at full speed
- Keep track of the time using variables on initiator side
- It waits for fixed time for logging (context switch between simulation kernel and thread will slightly slow down): report easy to understand
- This coding style is called **loosely-timed (LT)** in the TLM-2.0 standard

```
int sc_main(int argc, char* argv[])
{
  Top top("top");
  sc_start();
  return 0;
}
```

```
trans = { R, 20 } , data = aa000029 at time 0 s delay = 10 ns
trans = { R, 24 } , data = aa0000cd at time 10 ns delay = 10 ns
trans = { R, 28 } , data = aa0000ba at time 20 ns delay = 10 ns
trans = { R, 2c } , data = aa0000ab at time 30 ns delay = 10 ns
trans = { R, 30 } , data = aa0000f2 at time 40 ns delay = 10 ns
trans = { W, 34 } , data = ff000034 at time 50 ns delay = 10 ns
trans = { R, 38 } , data = aa0000e3 at time 60 ns delay = 10 ns
trans = { R, 3c } , data = aa000046 at time 70 ns delay = 10 ns
trans = { R, 40 } , data = aa00007c at time 80 ns delay = 10 ns
trans = { R, 44 } , data = aa0000c2 at time 90 ns delay = 10 ns
trans = { R, 48 } , data = aa000054 at time 100 ns delay = 10 ns
trans = { W, 4c } , data = ff00004c at time 110 ns delay = 10 ns
trans = { R, 50 } , data = aa00001b at time 120 ns delay = 10 ns
trans = { R, 54 } , data = aa0000e8 at time 130 ns delay = 10 ns
trans = { R, 58 } , data = aa0000e7 at time 140 ns delay = 10 ns
trans = { R, 5c } , data = aa00008d at time 150 ns delay = 10 ns
```

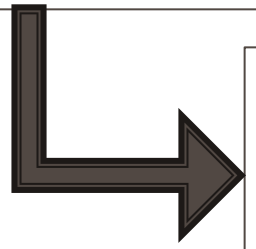
TLM Example: Summary

- TLM-2.0 ensures interoperability between models using:
  - a standard set of APIs
  - utility classes for consistent coding style.

Elements of interoperability

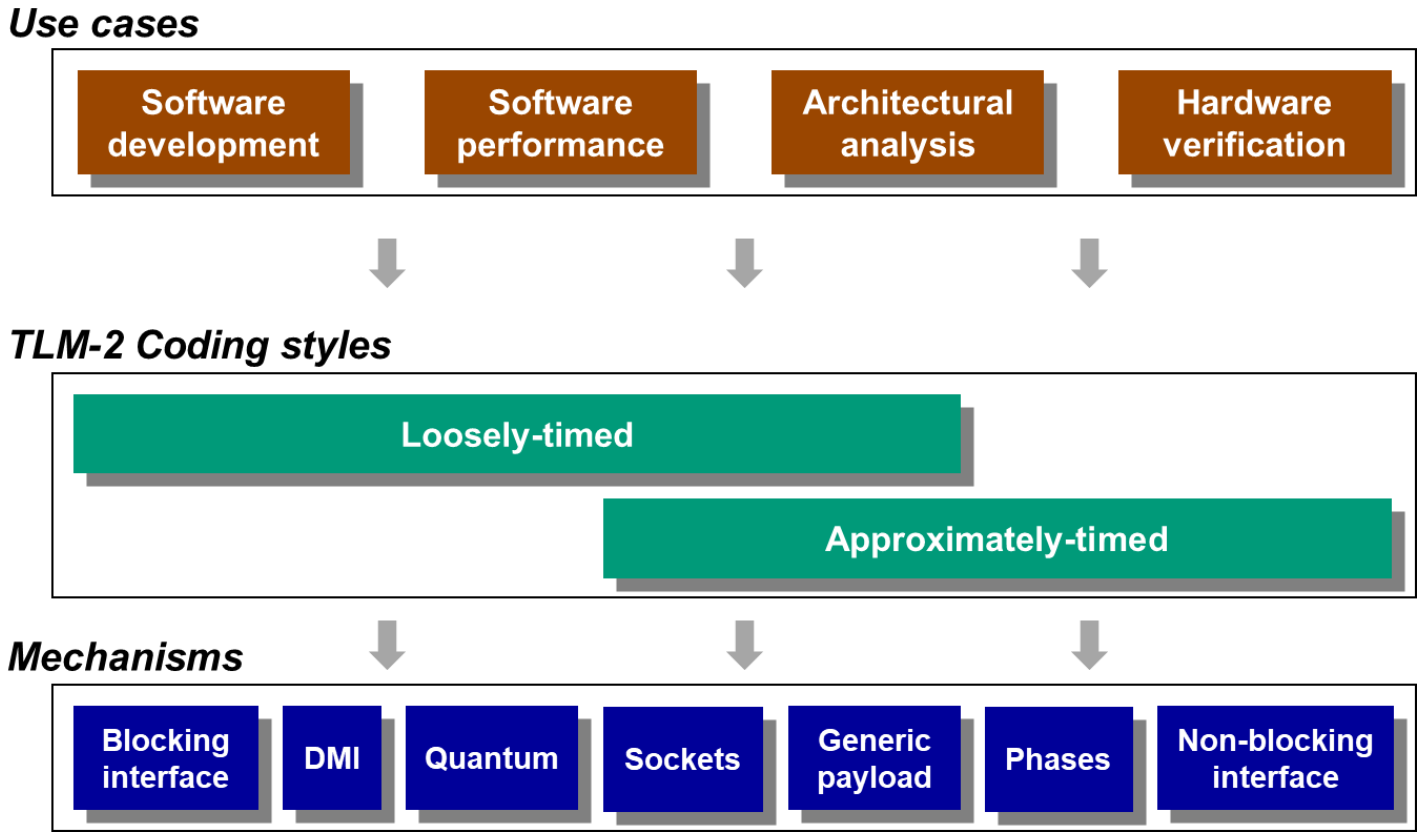
- Use of the standard initiator and target sockets
  - For each connection to a memory-mapped bus or other communication resource.
- Use of the generic payload
- Use of the base protocol: specifies the rules for using the generic payload with the standard TLM-2.0 interfaces and socket

Transaction-level Model



More Concepts

SystemC TLM: Overview



- Quantum
- DMI

https://systemc.org/overview/systemc-tlm/#resources

TLM: Temporal Decoupling

- Standard SystemC keeps a single synchronized view of time, which is used by all threads in all modules.

Temporal Decoupling:

- With temporal decoupling, initiator may run at a notional local time in advance of the current simulation time
  - Allows the thread to run ahead in simulation time, until it needs to synchronize with another thread.
- Initiator and target may each further advance the local time offset by increasing the value of the time argument
- Avoids bottlenecks in processing due to giving back control to the simulation kernel after each transaction (due to global time syncing)

TLM: Quantum

- Need to ensure that one thread doesn't run away hogging all the processing,
- Concept of the quantum defined in SystemC TLM

Quantum:

- A temporally decoupled initiator will continue to advance local time until the time quantum is exceeded
  - At that point, the initiator is obliged to synchronize by suspending execution, directly or indirectly calling the wait method with the local time as an argument.
- This allows other threads in the initiator a chance to catch up
- Ensures that the initiators execute in turn, each being responsible for determining when to hand back control by keeping track of its own local time.
- The original initiator should only run again after simulation time has advanced to the next quantum

TLM: Specialized Transactions

Debug transaction

- A debug transaction is a read that does not affect the state of the model.
- These are for use by debuggers, which wish to see the state of a model, without affecting that state.

Direct memory transactions

- A full-blown transaction is too heavyweight for some types of access.
- For example, an ISS accessing memory using transactions would destroy performance.
- Direct memory interface allows threads direct access to blocks of memory in other threads for high performance

References

- https://www.accellera.org/downloads/standards/systemc/files
- https://www.embecosm.com/appnotes/ean1/html/ch02.html
- https://www.doulos.com/knowhow/systemc/tlm-20/tutorial-1-sockets-generic-payload-blocking-transport/